

V8 Engine Internals

How JavaScript Actually Runs Under the Hood

A deep-dive guide covering the full pipeline: Parsing → AST → Bytecode → Optimized Machine Code — plus gotchas, hidden classes, and deoptimization.

Stage	Tool	Input	Output
Lexing	Scanner	Raw text	Tokens
Parsing	Parser	Tokens	AST
Interpretation	Ignition	AST	Bytecode
Optimization	TurboFan	Hot bytecode	Machine code
Deoptimization	TurboFan	Type violation	Back to bytecode

1. Parsing — Reading Your Code

Before V8 can do anything, it reads your source code as plain text and converts it into something it can reason about.

1A. Lexical Analysis (Tokenization)

The raw text is broken into **tokens** — the smallest meaningful units (keywords, identifiers, operators, literals).

```
const name = "Akshay";

// Tokens produced:

[const] [name] [=] ["Akshay"] [;]
↑ ↑ ↑ ↑ ↑
keyword ident op string semicolon
```

1B. Syntax Analysis

Tokens are checked against JavaScript grammar rules. This is where **SyntaxError** is thrown.

```
const = "Akshay"; // ■ SyntaxError – const cannot be an identifier
```

■ Eager vs Lazy Parsing

V8 uses two parsers to save startup time:

Parser	What it does	When used
Pre-parser	Skims code, skips function bodies	Functions not called immediately
Full parser	Deeply parses everything	Code that runs right away

```
// Lazy – function body SKIPPED until called
function heavyCalc() {
  const x = 10 * 20; // V8 barely looks at this initially
}

// Eager – runs immediately, fully parsed right away
console.log("hello");

// IIFEs are ALWAYS eagerly parsed
(function() {
```

```
// V8 fully parses this right away  
})();
```

■ Tip: IIFEs are always eagerly parsed because V8 knows they run immediately.

2. AST — Abstract Syntax Tree

After parsing, V8 builds a tree data structure representing your code's *meaning*, not its text. Think of it like a grammar diagram for your sentence.

```
const result = 2 + 3;

// AST structure:
VariableDeclaration (const)
  ■■■ VariableDeclarator
    ■■■ Identifier: "result"
      ■■■ BinaryExpression (+)
        ■■■ Literal: 2
        ■■■ Literal: 3
```

```
function greet(name) {
  return "Hello, " + name;
}

// AST structure:
FunctionDeclaration
  ■■■ Identifier: "greet"
  ■■■ Params: [Identifier: "name"]
  ■■■ BlockStatement
    ■■■ ReturnStatement
      ■■■ BinaryExpression (+)
        ■■■ Literal: "Hello, "
        ■■■ Identifier: "name"
```

■ Real-world connection: Babel, ESLint, and Prettier ALL work by reading the AST and transforming it. When you write a Next.js plugin or custom ESLint rule, you are working directly with ASTs. Try astexplorer.net to visualize any code.

3. Ignition — Bytecode Interpreter

The AST is handed to V8's interpreter called **Ignition**. It converts the AST into **bytecode** — a compact, lower-level instruction set.

Why not compile directly to machine code?

- Machine code takes significant time to generate
- Bytecode is fast to generate and small in size (critical for startup)
- Bytecode can be cached and reused across runs

```
function add(a, b) { return a + b; }  
add(1, 2);  
  
// Ignition bytecode (simplified):  
LdaNamedProperty a // Load 'a' into accumulator  
Add b // Add 'b' to accumulator  
Return // Return accumulator value
```

■ Real-world: Your Next.js app starts faster after the first load because V8 bytecode is cached on disk!

4. TurboFan — Optimizing JIT Compiler

V8 watches your code run. If a function is called many times ('hot function'), TurboFan compiles it to super-fast native machine code. This is called **JIT (Just-In-Time) compilation**.

```
function add(a, b) { return a + b; }

// Called thousands of times → V8 sends to TurboFan
for (let i = 0; i < 100000; i++) {
  add(1, 2);
}

// TurboFan assumes: a and b are ALWAYS numbers
// → compiles blazing-fast number-only machine code
```

■ Deoptimization — The Critical Gotcha

TurboFan optimizes based on **type assumptions**. If you suddenly break those assumptions, V8 throws away the optimized code and falls back to bytecode. This is called **deoptimization** (bailing out).

```
function add(a, b) { return a + b; }

// Phase 1: V8 sees only numbers → optimizes for numbers
for (let i = 0; i < 10000; i++) add(1, 2);

// Phase 2: You pass a string → DEOPTIMIZATION! ■
// TurboFan's compiled code is thrown away
add("hello", "world");
```

Deoptimization in Real React/Next.js Code

```
// ■ BAD – inconsistent object shapes cause deoptimization

function processUser(user) {
  return user.name + user.age;
}

processUser({ name: "Akshay", age: 25 });
processUser({ name: "Ravi" }); // missing age → different shape!
processUser({ age: 30, name: "Priya" }); // different property ORDER!
```

```
// ■ GOOD – consistent object shapes, same properties, same order  
processUser({ name: "Akshay", age: 25 });  
processUser({ name: "Ravi", age: 0 });
```

5. Hidden Classes — V8's Secret Weapon

V8 secretly assigns **hidden classes** (internal blueprints) to objects to speed up property access. Every time you add a property, V8 creates a new hidden class and transitions to it.

```
const obj = {}; // Hidden Class C0: {}  
  
obj.x = 10; // Hidden Class C1: { x }  
  
obj.y = 20; // Hidden Class C2: { x, y }
```

Adding properties in different orders → DIFFERENT hidden classes!

```
// ■ BAD — two different hidden class chains  
  
const p1 = new Point(1, 2); // Hidden class: { x, y }  
  
const p2 = {};  
p2.y = 5; // Hidden class: { y }  
p2.x = 10; // Hidden class: { y, x } ← DIFFERENT from p1!  
  
// ■ GOOD — always initialize in constructor, same order  
function Point(x, y) {  
  this.x = x; // always x first  
  this.y = y; // always y second  
}  
  
const a = new Point(1, 2); // same hidden class  
const b = new Point(3, 4); // same hidden class ■
```

■ This is exactly why React recommends initializing all state properties upfront in `useState` / `useReducer`!

6. Full Pipeline — Putting It All Together

Source Code: "const x = a + b"

■

▼

[Tokenizer] → const | x | = | a | + | b |

■

▼

[Parser] → AST: VariableDeclaration → BinaryExpression



[Ignition] → Bytecode (fast to generate, cacheable)



▼ (function called many times?)

[TurboFan] → Optimized Machine Code



■ (type mismatch?)

■■■■■■ Deoptimize ■■■

7. Rules to Write V8-Friendly Code

Rule	Why
Initialize all object properties in the constructor	Consistent hidden classes
Don't add / delete properties after creation	Avoids hidden class transitions
Keep function argument types consistent	Avoids deoptimization
Avoid delete obj.prop	Turns object into slow hash-table mode
Use arrays for same-type items	V8 uses typed array optimizations
Avoid the arguments object	Prevents Ignition optimizations

Key Takeaway

V8 bets on your types staying consistent. Write predictable, type-stable code and V8 rewards you with blazing-fast execution. Break that contract and it deoptimizes — you pay the performance penalty every time.